JAVA DSA – INFIX TO POSTFIX & PREFIX
FILL IN THE BLANK WORKSHEET

Based on the supplied C++ conversion programs | Career Credentials | Dr. Amar Panchal

Name: _____	Date: _____	Score: _____
Batch: _____	Time: _____	Mentor: Dr. Amar Panchal

Instructions

1. Fill every blank without changing the algorithm.

2. The precedence logic is intentionally kept from the supplied conversion programs.

3. Do not replace the precedence logic with a different algorithm.

4. Pay special attention to <= in Postfix and < in Prefix.

5. Complete the program first, then test with: A+B*C, (A+B)*C, and A+B*C-D.

Precedence Logic – Reference

```
if (ch == '+' || ch == '-')
    return 1;

if (ch == '*' || ch == '/' || ch == '%')
    return 2;

return 0;
```

Operator precedence: * / % > + -

1. Infix → Postfix – Fill in the Blanks

Keep the supplied precedence logic: while (!isEmpty() && prec(c) <= prec(s[tos]))

```
import java.util._____;

public class IntoPost {

    static char s[] = new char[100];
    static int tos, _____;

    static void createStack(int n) {
        maxsize = n;
        tos = _____;
    }

    static void push(char e) {
        tos++;
        s[_____] = e;
    }

    static char pop() {
        char temp = s[_____];
        tos--;
```

```
    return __________;
}

static boolean isEmpty() {
    return tos == ______;
}

static int prec(char ch) {
    if (ch == '+' || ch == ______)
        return ______;

    if (ch == '*' || ch == '/' || ch == ______)
        return ______;

    return ______;
}

public static void main(String[] args) {

    Scanner sc = new __________(System.in);

    char in[], post[] = new char[100];
    char c;
    int pi = ______;

    System.out.print("Enter infix: ");
    String input = sc.__________();

    in = input.__________();
    createStack(in.length);

    for (int i = 0; i < in.length; i++) {

        c = in[i];

        switch (c) {

            case '(':
                push(c);
                break;

            case ')':

                while (!isEmpty() && s[tos] != ______) {
                    post[pi] = ______();
                    pi++;
                }

                if (!isEmpty())
                    ______();

                break;

            case '+':
            case '-':
            case '*':
            case '/':
            case '%':
```

```
                    while (!isEmpty() &&
                           prec(c) ______ prec(s[tos])) {

                        post[pi] = ______();
                        pi++;
                    }

                    ______(c);
                    break;

                default:
                    post[pi] = ______;
                    pi++;
                    break;
            }
        }

        while (!__________()) {
            post[pi] = ______();
            pi++;
        }

        System.out.println(
            "Postfix is " + new String(post, 0, pi)
        );

        sc.close();
    }
}
```

2. Infix → Prefix – Fill in the Blanks

Keep the supplied precedence logic: scan right-to-left and use while (!isEmpty() && prec(c) < prec(s[tos]))

```
import java.util.__________;

public class IntoPre {

    static char s[] = new char[100];
    static int tos, __________;

    static void createStack(int n) {
        maxsize = n;
        tos = ______;
    }

    static void push(char e) {
        tos++;
        s[__________] = e;
    }

    static char pop() {
        char temp = s[__________];
        tos--;
        return __________;
    }

    static boolean isEmpty() {
        return tos == ______;
    }
```

```
static int prec(char ch) {
    if (ch == '+' || ch == ______)
        return ______;

    if (ch == '*' || ch == '/' || ch == ______)
        return ______;

    return ______;
}

public static void main(String[] args) {

    Scanner sc = new __________(System.in);

    char in[], pre[] = new char[100];
    char c;
    int pi = ______;

    System.out.print("Enter infix: ");
    String input = sc.__________();

    in = input.__________();
    createStack(in.length);

    // Scan from RIGHT to LEFT
    for (int i = in.length - 1; i >= 0; i--) {

        c = in[i];

        switch (c) {

            case ')':
                push(c);
                break;

            case '(':

                while (!isEmpty() && s[tos] != ______) {
                    pre[pi] = ______();
                    pi++;
                }

                if (!isEmpty())
                    ______();

                break;

            case '+':
            case '-':
            case '*':
            case '/':
            case '%':

                while (!isEmpty() &&
                       prec(c) ______ prec(s[tos])) {

                    pre[pi] = ______();
                    pi++;
                }
```

```
                    _____(c);
                    break;

                default:
                    pre[pi] = _____;
                    pi++;
                    break;
            }
        }

        while (!_________()) {
            pre[pi] = _____();
            pi++;
        }

        String prefix =
            new StringBuilder(
                new String(pre, 0, pi)
            )._________().toString();

        System.out.println(
            "Prefix is " + prefix
        );

        sc.close();
    }
}
```

3. Logic Check – Students Fill the Answers

1. In Postfix conversion, the expression is scanned from _____ to _____.

2. In Prefix conversion, the expression is scanned from _____ to _____.

3. In Postfix conversion, when an incoming operator has equal precedence, the supplied logic uses _____.

4. In Prefix conversion, the supplied logic uses _____ for the precedence comparison.

5. The precedence value of + and - is _____.

6. The precedence value of *, / and % is _____.

7. Opening bracket in Postfix is _____ onto the stack.

8. Closing bracket in Postfix causes operators to be _____ until '('.

9. In Prefix conversion, ')' is pushed and '(' causes popping until _____ is found.

10. At the end of either conversion, remaining stack operators are _____.

4. Practice Test

Q1. Convert A+B*C into Postfix and Prefix.

Q2. Convert (A+B)*C into Postfix and Prefix.

Q3. Convert A+B*C-D into Postfix and Prefix.

Q4. Convert (A+B)*(C-D) into Postfix and Prefix.